

Manual Técnico

Prototipo Web 3D Interactivo para Visualización Técnica de Drones

Alexander Woodcock Salomón

Diciembre 2025

Índice

1. Introducción	2
2. Arquitectura del Sistema	2
2.1. Stack Tecnológico	2
2.2. Diagrama de Componentes	2
3. Estructura del Proyecto (Unity)	2
4. Scripts Principales	2
4.1. GameManager.cs	2
4.2. AssetBundleLoader.cs	3
4.3. OrbitCamera.cs	3
5. Pipeline de Optimización (Technical Art)	3
5.1. Retopología	3
5.2. Baking de Texturas	3
6. Compilación y Despliegue (Build & Deploy)	3
6.1. Configuración de Build (Unity)	3
6.2. Generación de Asset Bundles	3
6.3. Despliegue en Servidor	4
7. Mantenimiento y Extensión	4
7.1. Añadir un Nuevo Modelo	4

1. Introducción

Este documento técnico detalla la arquitectura, componentes y procedimientos de mantenimiento del prototipo Web 3D para visualización técnica de drones. El sistema implementa más de 65 scripts con arquitectura modular, patrones de diseño profesionales y optimizaciones específicas para WebGL.

1.1. Alcance del Documento

- Arquitectura del sistema y patrones de diseño
- Descripción de todos los sistemas implementados
- Procedimientos de compilación y despliegue
- Guías de extensión y mantenimiento

2. Arquitectura del Sistema

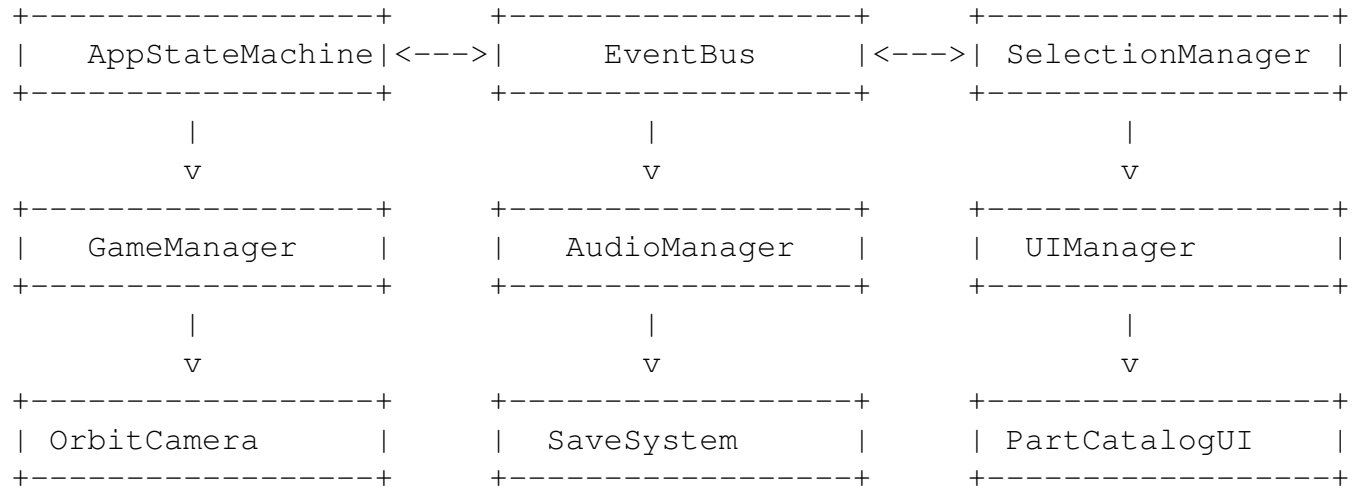
2.1. Stack Tecnológico

- **Motor:** Unity 6.3 LTS
- **Lenguaje:** C# 11 (Scripting) / HLSL (Shaders)
- **Plataforma:** WebGL 2.0
- **Compilador:** IL2CPP → WebAssembly (WASM)
- **Pipeline Gráfico:** Universal Render Pipeline (URP)
- **UI Framework:** UI Toolkit (USS/UXML)

2.2. Patrones de Diseño Implementados

- **Singleton/PersistentSingleton:** Para managers globales
- **Event Bus:** Comunicación desacoplada entre sistemas
- **State Machine:** Control de estados de la aplicación
- **ScriptableObjects:** Configuración de datos (DronePartData, AssemblyGuideData)
- **Object Pooling:** Reutilización de objetos para rendimiento

2.3. Diagrama de Arquitectura



3. Estructura del Proyecto

```

Assets/Scripts/
+-- Core/
|   +-- Data/           # ScriptableObjects (DronePartData)
|   +-- Events/         # Sistema de eventos (EventBus)
|   +-- Managers/       # 25+ Managers (Singletons)
|   +-- Content/        # Componentes de piezas
|   +-- Utils/          # Utilidades (Singleton, TweenEngine)
+-- UI/
|   +-- UIManager.cs    # Control principal UI
|   +-- PartCatalogUI   # Catálogo de piezas
|   +-- EnhancedInfoPanel # Panel de información
|   +-- AssemblyStepUI  # Guía de ensamblaje
+-- Graphics/
|   +-- Shaders/        # Shaders personalizados
|   +-- PostProcessing/ # Efectos de post-procesado

```

4. Sistemas Principales

4.1. Core Managers (25+ Scripts)

4.1.1. AppStateMachine.cs

Máquina de estados principal. Estados: Loading, Exploration, ExplodedView, PartFocus, Settings, Menu.

```
AppStateMachine.Instance.TransitionTo(AppState.ExplodedView);
```

4.1.2. EventBus.cs

Sistema de eventos pub/sub para comunicación desacoplada.

```
EventBus.Publish(new PartSelectedEvent(partData));  
EventBus.Subscribe<PartSelectedEvent>(OnPartSelected);
```

4.1.3. SelectionManager.cs

Gestiona selección de piezas con raycast, integra HighlightSystem, CursorManager y AnalyticsManager.

4.1.4. ViewModeManager.cs

Controla 7 modos de visualización:

- Realistic - Material original
- XRay - Semi-transparente con fresnel
- Blueprint - Estilo técnico
- SolidColor - Color sólido configurable
- Wireframe - Solo líneas
- Ghosted - Fantasma
- Thermal - Vista térmica

4.1.5. PartCatalogManager.cs

Búsqueda y filtrado de piezas por nombre, tipo, categoría y material.

4.1.6. PartVisibilityManager.cs

Control de visibilidad: ocultar, mostrar, aislar piezas con animaciones de fade.

4.1.7. CrossSectionManager.cs

Cortes transversales en ejes X, Y, Z con plano visual y posición configurable.

4.1.8. DroneStateController.cs

Control de estado del dron: Off, StartingUp, Idle, Flying, ShuttingDown. Incluye animación de propelas, luces de estado y partículas.

4.1.9. ModularPartsSystem.cs

Sistema de slots para intercambio de piezas modulares con animaciones.

4.2. Herramientas de Ensamblaje

4.2.1. AssemblyGuideManager.cs

Guía paso a paso con:

- Pasos secuenciales con navegación
- Herramientas requeridas por paso
- Tiempos estimados
- Niveles de dificultad (1-5)
- Advertencias de seguridad
- Tips de instalación

4.2.2. MeasurementTool.cs

Herramienta de medición con modos: Distancia, Ángulo, Radio.

4.2.3. ConnectionPointsViewer.cs

Visualización de puntos de conexión (tornillos, snaps, soldaduras, cables).

4.2.4. BillOfMaterialsManager.cs

Generación de lista de materiales con exportación CSV.

4.2.5. AnnotationSystem.cs

Sistema de notas 3D con billboard text.

4.2.6. AssemblyChecklist.cs

Lista de verificación de piezas con progreso.

4.3. Sistemas de Audio y Visual

4.3.1. AudioManager.cs

Control de audio con volúmenes separados (Master, SFX, Music) y persistencia.

4.3.2. QualityManager.cs

Ajuste dinámico de calidad basado en FPS.

4.3.3. TweenEngine.cs

Motor de animaciones ligero (alternativa a DOTween) con 8 tipos de easing.

4.3.4. NotificationManager.cs

Sistema de notificaciones toast animadas.

5. ScriptableObjects

5.1. DronePartData

Configuración completa de cada pieza:

- Info General: nombre, ID, tipo, descripción
- Specs Técnicos: peso, dimensiones, función
- Materiales: tipo, propiedades, fabricante
- Vista Explosionada: dirección, distancia, prioridad
- Info de Ensamblaje: herramientas, torque, tornillos, dificultad

5.2. AssemblyGuideData

Definición de guías de ensamblaje con lista de pasos configurables.

6. Compilación y Despliegue

6.1. Requisitos de Build

1. Unity 6.3 LTS
2. WebGL Build Support instalado
3. URP configurado

6.2. Configuración de Build

1. File > Build Settings > WebGL
2. Player Settings:
 - Compression: Brotli (producción) o Disabled (desarrollo)
 - Code Stripping: High
 - IL2CPP Code Generation: Faster (Smaller) Builds

6.3. Despliegue

Subir carpeta Build/ a servidor con headers:

```
Content-Type: application/wasm (para .wasm)
Content-Encoding: br (si usa Brotli)
```

7. Extensión del Sistema

7.1. Añadir Nueva Pieza

1. Crear DronePartData asset (Create > WebGL > Drone Part Data)
2. Configurar todos los campos
3. Añadir ExplodablePart component al modelo 3D
4. Asignar DronePartData al component
5. Añadir ConnectionPointMarker a puntos de conexión

7.2. Añadir Nuevo Modo de Vista

1. Añadir entrada a enum ViewMode
2. Crear material en ViewModeManager
3. Implementar lógica en ApplyModeToRenderers()
4. Añadir botón en ViewModeToolbar

7.3. Añadir Nuevo Manager

1. Heredar de Singleton;T¿o PersistentSingleton;T¿
2. Implementar lógica en Awake() y Start()
3. Suscribirse a eventos relevantes del EventBus
4. Añadir al GameObject _GameManagers en escena

Script	Función
--------	---------

8. Resumen de Scripts

Script	Función
AppStateMachine	Máquina de estados principal
EventBus	Sistema de eventos
SelectionManager	Selección de piezas
ViewModeManager	Modos de visualización (7)
PartCatalogManager	Búsqueda y filtrado
PartVisibilityManager	Ocultar/mostrar piezas
CrossSectionManager	Cortes transversales
DroneStateController	On/Off del dron
ModularPartsSystem	Intercambio de piezas
AssemblyGuideManager	Guía de ensamblaje
MeasurementTool	Mediciones 3D
ConnectionPointsViewer	Puntos de conexión
BillOfMaterialsManager	Lista de materiales
AnnotationSystem	Notas 3D
AssemblyChecklist	Verificación de piezas
AudioManager	Control de audio
TweenEngine	Motor de animaciones
OrbitCameraController	Cámara orbital
CameraPresets	Vistas predefinidas
HighlightSystem	Resaltado visual
CursorManager	Cursores dinámicos
LoadingController	Pantalla de carga
AnalyticsManager	Métricas de uso
AccessibilityManager	Opciones de accesibilidad
ErrorHandler	Manejo de errores
ScreenshotManager	Captura de pantalla
KeyboardShortcuts	Atajos de teclado
PerformanceMonitor	Métricas de rendimiento
WebGLOptimizer	Optimizaciones WebGL

Total: 65+ scripts, 9,000+ líneas de código